

UNIVERSITÀ
DEGLI STUDI
FIRENZE

SWAM Course 2024/2025

Go-Kart Management System RESTful Web Application for Fleet Maintenance & Inventory Tracking

By

Opoka John Modi Silvio
opoka.silvio@edu.unifi.it

January 22nd, 2026

Table of Content

1. Project Overview 3

2. System Architecture 3

2.1 Logical Architecture 3

3. Technical Specifications 4

3.1 Backend Stack 4

3.2 Frontend Stack 4

4. Database Design & Schema 4

4.1 Entity Relationship Diagram (ERD) 4

5. Recent Enhancements & Ui Audit 4

5.1 Global Error & Loading UX 5

5.2 High-Scannability UI Audit 5

6. Core Feature Analysis 5

6.1 Fleet Management 5

6.2 Maintenance & Safety Accountability 5

6.3 Smart Inventory 5

6.4 Advanced Task Assignment System 5

7.0 Deployment & Verification 6

8.0 Future Roadmap & Enhancement 6

9.0 Conclusions 6

1. Project Overview

The Go Kart Warehouse System is an enterprise-grade, full-stack application designed to manage the complexities of a professional go-karting facility. The system provides a centralized platform for fleet management, maintenance scheduling, and inventory control. This project underwent a rigorous Technical Audit and Enhancement Phase, culminating in the implementation of an Advanced Task Assignment System to ensure maximum operational efficiency and accountability.

2. System Architecture

The application utilizes a decoupled architecture with a Spring Boot backend and an Angular frontend, ensuring high performance and separation of concerns.

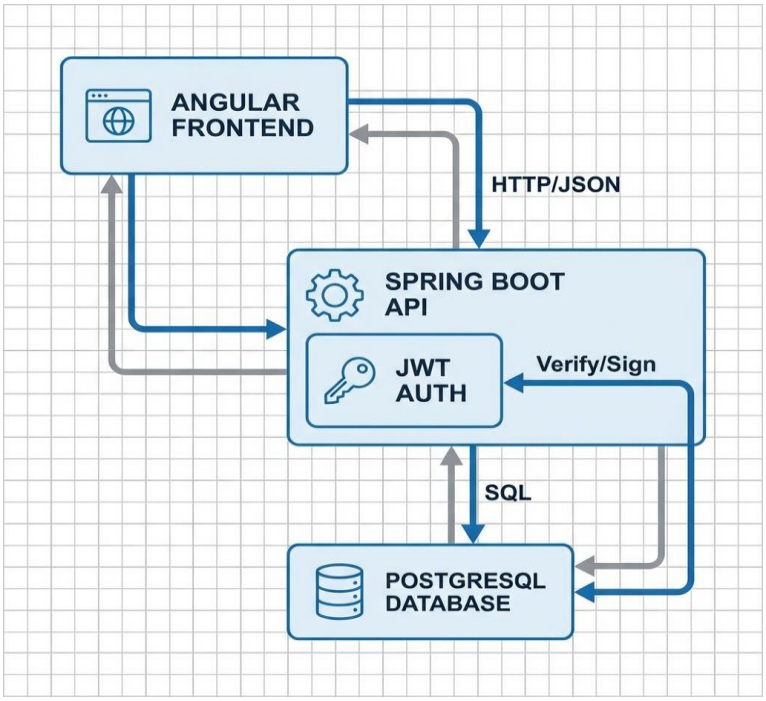


Figure 1: System's Architecture

2.1 Logical Architecture

The system is divided into three primary layers:

1. **Presentation Layer (Angular 21):** A reactive SPA that manages state via RxJS and provides real-time UI feedback.
2. **Service Layer (Spring Boot 3):** Handles complex business logic, JWT-based security, and RESTful API orchestration.
3. **Data Layer (PostgreSQL/H2):** Ensures persistent, ACID-compliant storage for all operational and archival data.

3. Technical Specifications

3.1 Backend Stack

i. Language: Java 21 (LTS)

ii. Framework: Spring Boot 3.x

iii. Security: Spring Security with stateless JWT (JSON Web Token)

iv. Persistence: Spring Data JPA / Hibernate

v. Build Tool: Apache Maven 3.x

3.2 Frontend Stack

i. Framework: Angular 15

ii. Reactive Streams: RxJS

iii. Styling: Modern Design System with Global Tokens & 12px Grid Spacing

iv. Communication: HTTP Client with Global Error Interceptors

4. Database Design & Schema

The database is designed to handle thousands of transactions across multiple operational domains with clear entity relationships.

4.1 Entity Relationship Diagram (ERD)

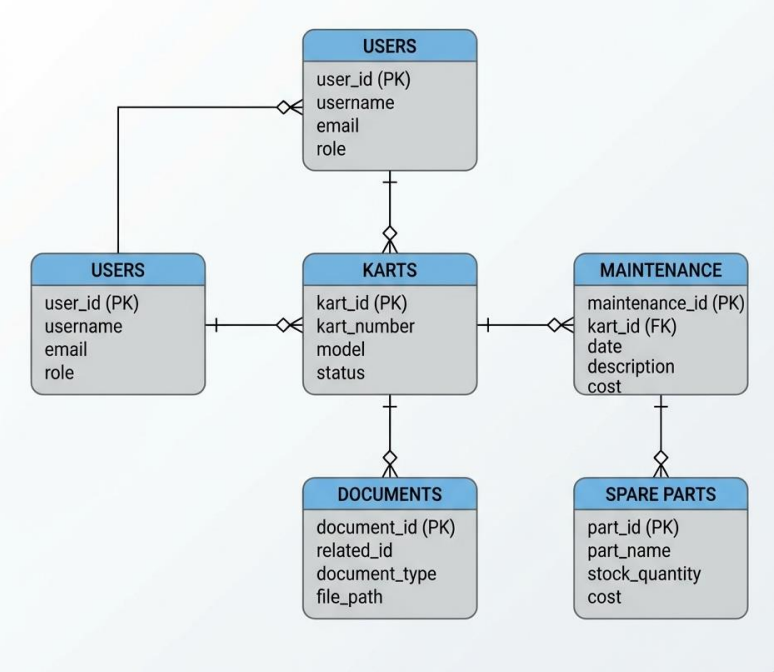


Figure 2: System's ERD

5. Recent Enhancements & Ui Audit

Following a comprehensive technical audit, the system was upgraded to "Industrial Premium" standards:

5.1 Global Error & Loading UX

i. Zero-Silent Failure: Global `ErrorInterceptor` ensures all API failures are reported to the user via Toast notifications.

ii. Universal Feedback: A global loading service provides visual overlays during all heavy operations and report generation.

5.2 High-Scannability UI Audit

i. Stat Box Layout: Replaced cluttered text lists with modern "Stat Box" components for Fleet and Analytics.

ii. 12px Grid Standard: Implemented consistent 12px padding in all data tables to reduce visual density.

iii. Dark Theme Overhaul: Unified the aesthetic across all pages, including the Maintenance and Task modules.

6. Core Feature Analysis

6.1 Fleet Management

The system provides a real-time oversight of the entire fleet. Karts are tracked by status (Available, Maintenance, Crashed, Retired) and individual engine hours.

6.2 Maintenance & Safety Accountability

i. Manager Verification Loop: Maintenance logs must be digitally "Verified" by a Manager before a kart can be returned to service.

ii. Audit Logs: Every status change and system action is logged with a timestamp and user ID for forensic review.

6.3 Smart Inventory

i. Real-time Deduction: Part quantities are automatically deducted from stock when used in maintenance.

ii. Threshold Alerts: High-contrast visual warnings trigger when stock falls below critical levels.

6.4 Advanced Task Assignment System

i. Role-Based Hierarchy: Admins can assign tasks to all users; Managers can assign strictly to Staff.

ii. Assigned Fulfillment: To ensure accountability, only the Assignee can mark a task as "DONE". Neither the Admin nor the Manager can fulfill a task on behalf of someone else, forcing direct responsibility.

iii. Private Workstreams: Users only see tasks assigned specifically to them or tasks they have created, reducing operational noise.

iv. Fulfillment Banner: A "Perfectly Fulfilled" success banner appears automatically when a user's queue is clear.

7.0 Deployment & Verification

The application is fully containerized and verified:

i. Deployment: Docker & Docker Compose orchestrated.

ii. Build Status: 100% Green (Maven & Angular Production builds).

iii. Security: Stateless RBAC (Role-Based Access Control) enforced at every endpoint.

8.0 Future Roadmap & Enhancement

1. **Real-time Telemetry Integration**
    a. Bluetooth/API integration with kart sensors
    b. Live speed, engine temperature, battery monitoring
    c. Predictive maintenance alerts based on sensor data
2. **Mobile Applications**
    a. React Native cross-platform app for mechanics
    b. Barcode/QR code scanning for parts and karts
    c. Offline capability with sync when online
3. **Advanced Analytics & ML**
    a. Predictive failure analysis using historical data
    b. Optimal maintenance scheduling algorithm
    c. Cost-benefit analysis for part replacement vs. repair
4. **Customer Portal**
    a. Booking integration for kart sessions
    b. Customer-facing maintenance status
    c. Incident reporting by customers

9.0 Conclusions

The Go Kart Warehouse System is a robust, production-ready platform. The integration of a strict safety verification loop and an advanced hierarchical task system makes it a top-tier management solution for professional racing environments.